



CS401 MPP

W3D4 Overview

The Stream API (part 2)

Overview of Today and Tomorrow

- Intro to Java 8 Streams
- Streams Basic Facts
- Ways to Create Streams
- Intermediate Operations
- Stateful Intermediate Operations
- Terminal Operations
- Working with Optional
- Reduce
- Collecting Results
- Primitive Type Streams
- Lambda Libraries



CS401 MPP

The Stream API

Terminal Operations

Terminal Operations

- The last step in a pipeline of Streams is an operation that produces a final output
 - These are called *terminal operations* because, once they are called, the stream can no longer be used
 - They are also called *reduction methods* because they reduce the Stream to some final value

Terminal Operations

- We have already seen one example:
 - `.collect(Collectors.toList())`
- Use **`.count()`** to count the number of elements

```
List<String> words = //...
long numLongWords
    = words.stream()
            .filter(w -> w.length() > 12)
            .count();
```

Terminal Operations

- `max()`, `min()`, `findFirst()`, `findAny()`
 - Search a stream for particular values
 - Will throw an exception if not handled properly

```
Optional<String> largest = words.stream()
    .max(String::compareToIgnoreCase);

if (largest.isPresent()) {
    System.out.println("largest: " + largest.get());
}
```

Example usage of `.max()`

Optional

- An **Optional** is a wrapper for the answer
 - either the found String can be read via **get()**
 - or a boolean flag can be read that says no value was found
 - (for example, if the **Stream** was empty).
- You can call **get()** on an **Optional** to retrieve the stored value
 - but if the value was not found it produces a **NoSuchElementException**.
 - That's why it's good to check that **isPresent()** before **get()**

findFirst() & findAny()

- **findAny()** returns an **Optional** that contains some value in an input **Stream**
 - How this value is chosen from the **Stream** is not guaranteed
 - If the **Stream** is empty, the returned **Optional** is empty
 - This operation works much better with parallel **Streams** than **findFirst**

```
Optional<String> startsWithQ  
    = words.stream()  
        .filter(s -> s.startsWith("Q"))  
        .findFirst();
```

```
Optional<String> startsWithQ  
    = words.parallelStream()  
        .filter(s -> s.startsWith("Q"))  
        .findAny();
```




CS401 MPP

The Stream API

Working with Optional

A Better Way to Handle Nulls

- **Optional** was introduced in the previous video
 - It was added to make handling **nulls** less error prone
 - However notice:

```
if (optionalValue.isPresent()) {  
    optionalValue.get().someMethod();  
}
```

- Is no easier than:

```
if (value != null) {  
    value.someMethod();  
}
```

The Optional class, however, supports other techniques that are superior to checking nulls.

orElse()

- The **orElse** method
 - Provides alternative output
 - When the result is null

```
//OLD WAY
public static void pickName(final List<String> names, final String startingLetter) {
    String foundName = null;
    for (String name : names) {
        if (name.startsWith(startingLetter)) {
            foundName = name;
            break;
        }
    }
    System.out.print(String.format("A name starting with %s: ", startingLetter));
    if (foundName != null) {
        System.out.println(foundName);
    } else {
        System.out.println("No name found");
    }
}
```

orElse()

//NEW WAY

```
public static void pickName(final List<String> names, final String startingLetter) {  
    final Optional<String> foundName = names.stream()  
                                            .filter(name -> name.startsWith(startingLetter))  
                                            .findFirst();  
  
    System.out.println(  
        String.format("A name starting with %s: %s",  
            startingLetter,  
            foundName.orElse("No name found")));  
}
```

- The **orElse** method on an **Optional**
 - Returns the value stored in the **Optional**, if it is present
 - If not present, returns the alternative value supplied as an argument to **orElse**
- The alternative value supplied must have the same type
 - as the value stored in the original **Optional**

ifPresent (Consumer)

- ifPresent(Consumer) invokes an action
 - Only when the value is present
 - Skips the null case completely

This is an overloaded version of
ifPresent()

```
public static void pickName(List<String> names, String startingLetter) {  
  
    final Optional<String> foundName  
        = names.stream()  
            .filter(name -> name.startsWith(startingLetter))  
            .findFirst();  
  
    foundName.ifPresent(  
        name -> System.out.println("Hello " + name));  
}
```

If name is found it is printed
if not, nothing happens

Creating Your Own Optionals

- You can create an Optional instance in your own code using the static method **of()**
 - However, if of is used on a null value, a **NullPointerException** is thrown,
 - Best practice is to use **of()** together with **empty()**

```
public static Optional<Double> inverse(Double x) {  
    return x == 0 ? Optional.empty() : Optional.of(div(1d, x));  
}  
  
// returns a/b if possible  
private static Double div(Double a, Double b) {  
    if (b == 0) {  
        return null;  
    }  
    return a / b;  
}
```

Optional.empty() creates an **Optional** with no wrapped value this means that **isPresent()** is false

Creating Your Own Optionals

- The static method **ofNullable()**
 - lets you read in a possibly **null** value
 - Specifically **ofNullable()** returns an **Optional** that embeds the specified value if non-null
 - Otherwise returns an empty **Optional**.

```
public String createOutput() {  
    String outputMessage = "Hello ";  
    outputMessage += Optional.ofNullable(readInput()).orElse("World!");  
    return outputMessage;  
}
```

You can use **orElse()** / **orElseGet()** together with **Optional.ofNullable()**

Where **readInput()** returns either null or a person's name

ofNullable with orElse / orElseGet

- **OrElseGet()** is for getting a value from a method call
 - **orElse()** is for (values / property / local var)
 - Example: a lazy singleton made with **orElseGet()**

```
private static Connection conn = null;
private static Connection createConnection() {
    try {
        conn = DriverManager.getConnection(DB_URL, USERNAME, PASSWORD);
        return conn;
    } catch (SQLException e) {
        throw new RuntimeException(e);
    }
}
public static Connection getConnection() {
    return Optional.ofNullable(conn).orElseGet(ConnectManager::createConnection);
}
```


orElseGet(Supplier)

- **orElseGet** expects a **Supplier** argument

```
interface Supplier<T> {  
    T get();  
}
```

orElse VS orElseGet

- If we had accidentally used **orElse** in the previous example:

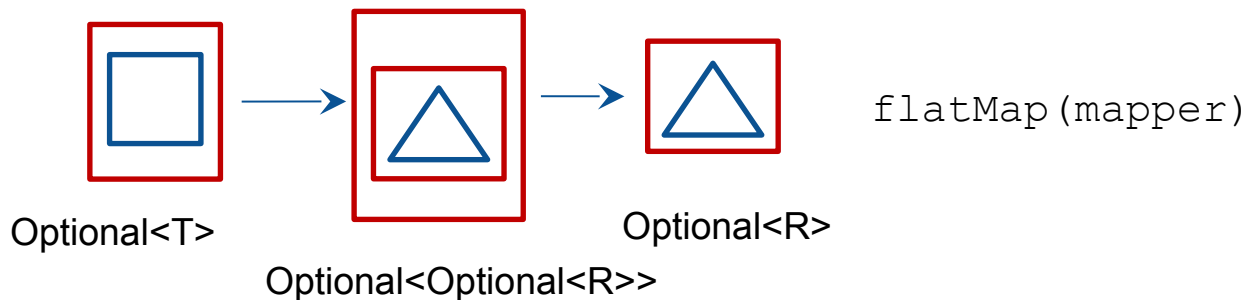
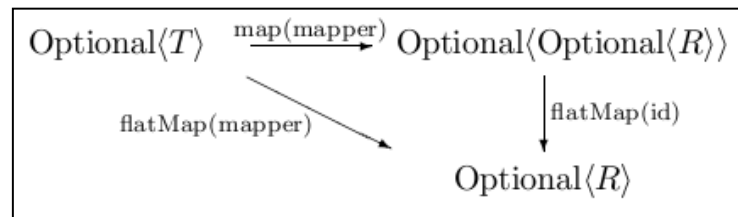
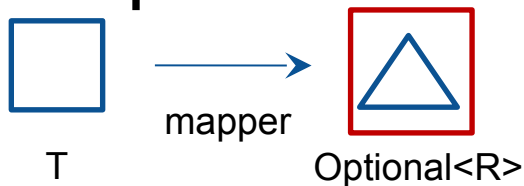
```
public Connection getConnection() {  
    return Optional.ofNullable(conn).orElse(createConnection());  
}
```

We have to make a method call to satisfy the compiler (which expects a value)

- Call to **createConnection()** is made regardless of what the **Optional** is
 - If it is null, call is made and return value is used
 - If it is not null, call is made and value is ignored
- But **orElseGet()** only executes its method reference
 - when **Optional** is null
 - Not null, nothing is executed

Using Optional: flatMap

- The **flatMap** method works the same way as on Streams
 - Allows you to chain method calls and skip null checks
 - Given a mapper from type **T** to an **Optional<R>**, flatMap transforms an **Optional<T>** first to an **Optional<Optional<R>>**, then to an **Optional<R>**



Example

Imperative Solution

- At most one company is a winner in a multi-company contest.

- The winning company will have one or more employees with a winning ticket.
- Find the primary telephone number of the employee with the biggest winnings.

```
private static String telNumberOfBiggestWinnerImperative(List<Company> list) {
    boolean winningCompanyFound = false;
    for(Company c: list) {
        if(c.hasWinningTicket()) {
            winningCompanyFound = true;
            int largestSoFar = 0;
            Employee biggestWinner = null;
            List<Employee> emps = c.getEmployees();
            for(Employee e: emps) {
                int wins = e.getWinningAmount();
                if(wins > largestSoFar) {
                    largestSoFar = wins;
                    biggestWinner = e;
                }
            }
            List<String> telNums = biggestWinner.getTelephoneNumbers();
            if(telNums != null && telNums.size() > 0)
                return telNums.get(0);
            else
                return "Winner has no phone number";
        }
    }
    if(!winningCompanyFound) {
        return "No winning company";
    }
    return null;
}
```

Solution with Optional.flatMap()

```
static Comparator<Employee> winnings
    = Comparator.comparing((Employee e) -> e.getWinningAmount());

private static String telNumberOfBiggestWinner(List<Company> list) {
    return list.stream().filter((Company c) -> c.hasWinningTicket())
        .findAny()
        .flatMap((Company c) -> c.getEmployees().stream().max(winnings))
        .flatMap((Employee e) -> e.getTelephoneNumbers().stream().findFirst())
        .orElse("Not found");
}
```

Replacing Null Checks

- Consider the following imperative code
 - Checks if a Person from Fairfield is in a list of Persons

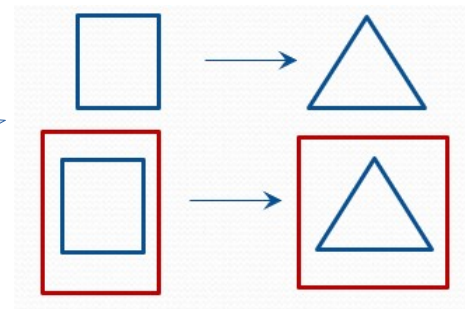
```
private static boolean personFromFairfield(List<Person> persons) {  
    Person foundPerson = null;  
    for(Person p: persons) {  
        if(p != null) {  
            Address addr = p.getAddress();  
            if(addr != null) {  
                String city = addr.getCity();  
                if(city != null) {  
                    if(city.equals("Fairfield")) {  
                        foundPerson = p;  
                    }  
                }  
            }  
        }  
    }  
    return foundPerson != null;  
}
```

Eliminating Null Checks

- Use **Optionals** with `map()` to eliminate

```
private static boolean personFromFairfield(List<Person> persons) {  
    for(Person p: persons) {  
        if(Optional.ofNullable(p).map(x -> x.getAddress())  
            .map(x->x.getCity())  
            .orElse("").equals("Fairfield")) {  
            return true;  
        }  
    }  
    return false;  
}
```

If an Optional opt is empty
`opt.map(mapper)` returns
an empty Optional





CS401 MPP

The Stream API

Reduce

Reduce

- **.reduce()** is another terminal operation
 - Lets you combine the elements of a Stream
 - Into a single value

By repeatedly applying an operation

- Example: sum of digits

```
int sum = 0;
for(int x : digits) {
    sum += x;
}
```

Procedural code

```
Integer sum2 = digits.stream().reduce(0, (x, y) -> x + y);
```

Using **reduce**

Syntax

```
digits.stream().reduce(0, (x, y) -> x + y);
```

- First arg: Initial Value
 - The value that is returned if the stream is empty
 - The starting point for the computation
 - Should be the *identity element* for the combining operation
- Second arg: BinaryOperator Lambda

```
interface BinaryOperator<T> {  
    T apply(T a, T b);  
}
```

The Reduce Computation

- The computation starts by applying the binary operator to the pair (**e**, **f**)
 - where **e** is the initial value (0 in our example)
 - **f** is the first element of the stream,
 - This produces **accum**
- Then the binary operator is applied to (**accum**, **g**)
 - Where **g** is the next element in the stream
 - The result is stored in **accum** again
 - This continues until all stream values have been used.

Example

- Starting with the following stream: **[2,1,4,3]**
 - Using ordinary addition for the **BinaryOperator**
 - Then reduce performs the following computation
$$((0 + 2) + 1) + 4 + 3 = 10$$
- Parallel computation can improve performance
 - **[2, 1, 4, 3]** is broken up into **[2, 3], [4, 1]**
 - Then in parallel we arrive at the same answer:
$$\text{sum1} = (0 + 2) + 3 \qquad \text{sum2} = (0 + 4) + 1$$
$$\text{combined} = \text{sum1} + \text{sum2} = 10$$

Product of Numbers

- To form the product of a list of numbers
 - For the initial value ask: “What is the identity element for the multiplication operation?”
 - The identity is 1. (Note that, by convention, the product of an empty list of numbers is 1)

```
int product = numbers.stream().reduce(1, (x, y) -> x * y);
```

The code

Subtraction

- What happens this code is executed?

- Try it with the list [2, 1, 4, 3]

0 is not actually an identity for subtraction
but it does not matter in this scenario

```
int difference= numbers.stream().reduce(0, (a, b) -> a - b);
```

- Computation produces:
 - $((0 - 2) - 1) - 4) - 3$ //output: -10
 - Parallel computation produces:
 - $\text{diff1} = (0 - 2) - 3$ $\text{diff2} = (0 - 1) - 4$
 - $\text{combined} = \text{diff1} - \text{diff2}$ // output: 0
 - Therefore:
 - Use reduce only on operations that are commutative and associative.

+ and * are commutative & associative
but subtraction is not

Reduce Overloaded

- There is a version of reduce with only 1 arg:

```
Optional<Integer> sum = digits.stream().reduce((x, y) -> x + y);
```

- This version produces an **Optional<T>**
 - Contains the same result if **Stream** is nonempty
 - When the **Stream** is empty it contains **null**
 - In other words, no value is present in the **Optional**

Exercise 9.4

- Use **reduce** to concatenate the Strings in the Stream below to form a single, space-separated **String**
 - Print the result to the console
 - See lesson9.exercise_4 in the InClassExercises

```
public static void main(String[] args) {  
    Stream<String> strings = Stream.of("A", "good", "day", "to", "write", "some", "Java");  
}
```


Main Point 4

- When a Collection is wrapped in a Stream
 - it becomes possible to rapidly make transformations and extract information in ways that would be much less efficient, maintainable, and understandable without the use of Streams.
 - In this sense, Streams in Java represent a deeper level of intelligence inherent in the concept of “collection” that has been implemented in the Java language.
- When intelligence expands, challenges and tasks that seemed difficult and time-consuming before can become effortless and meet with consistent success.
 - This is one of the documented benefits of TM practice.



CS401 MPP

The Stream API

Collecting Results

Collecting Results

- One kind of terminal operation is a reduction that outputs a single value, like **max** or **count** or **reduce**.
 - Another kind of terminal operation collects the elements into a type of collection, like an **array**, **list**, or **map**.

```
String[] result = stream.toArray(String[]::new);
```

```
List<String> result = stream.collect(Collectors.toList());
```

```
Set<String> result = stream.collect(Collectors.toSet());
```

```
TreeSet<String> result = stream.collect(Collectors.toCollection(TreeSet::new));
```

Works for any type of specific implementation
such as ArrayList, HashMap

Collecting into a String

```
String result = stream.collect(Collectors.joining());
```

Collect all the strings in a stream by concatenating them

```
String result = stream.collect(Collectors.joining(", "));
```

Separate strings by commas

```
String result = stream.map(Object::toString)  
    .collect(Collectors.joining(", "));
```

Turn objects into strings before joining

Note: instead of `Object::toString` you can use your own object type like `Employee::toString`. By polymorphism, either way works
See demo `lesson9.lecture.collect`

Collecting into a Map

These examples use `personStrm`, which is a `Stream<Person>`

```
Map<Integer, String> idToName =  
    personStrm.collect(Collectors.toMap(Person::getId, Person::getName));
```

Key = id, Value = name

```
Map<Integer, Person> idToPerson =  
    personStrm.collect(Collectors.toMap(Person::getId, Function.identity()));
```

Key = id, Value = the Person object

Note: **`identity()`** is a static method on `Function` that returns a function that always returns its input argument. In the example, it is the function:
`(Person p) -> p`

Summary Statistics

- Summary statistics can be made for Streams whose elements can map to: **Int**, **Double**, or **Long**
 - Provides sum, average, maximum, and minimum

```
IntSummaryStatistics summary =  
    words.collect(Collectors.summarizingInt(String::length));  
double averageWordLength = summary.getAverage();  
double maxWordLength = summary.getMax();
```

- DoubleSummaryStatistics uses Collectors.summarizingDouble
- LongSummaryStatistics uses Collectors.summarizingLong

ToIntFunction

- **IntSummaryStatistics** extracts int info from a **Stream**
 - The elements of the **Stream** will be converted at runtime to (primitive) **ints** for **summarizingInt** to perform its tasks.
 - The **summarizingInt** method expects an argument that is an implementation of the **ToIntFunction<T>** interface:

```
interface ToIntFunction<T> {  
    int applyAsInt(T value);  
}
```

Exercise 9.5

- Use **DoubleSummaryStatistics** to output the top test score, lowest test score, and average among all test scores in a given list
 - See lesson9.exercise_5 in InClassExercises

```
public class ExamData {  
    private String studentName;  
    private double testScore;  
    public ExamData(String name, double score) {  
        studentName = name;  
        testScore = score;  
    }  
}
```

```
public static void main(String[] args) {  
    List<ExamData> data = new ArrayList<ExamData>() {  
        {  
            add(new ExamData("George", 91.3));  
            add(new ExamData("Tom", 88.9));  
            add(new ExamData("Rick", 80));  
            add(new ExamData("Harold", 90.8));  
            add(new ExamData("Ignatius", 60.9));  
            add(new ExamData("Anna", 77));  
            add(new ExamData("Susan", 87.3));  
            add(new ExamData("Phil", 99.1));  
            add(new ExamData("Alex", 84));  
        }  
    };  
}
```


Main Point 5

- To obtain some information about a collection, we can lift the collection to a Stream, perform transformations at the Stream level, and then, by way of some terminal operation, collapse the Stream to obtain a final output. It is by virtue of accessing the powerful dynamics available in Stream that the result is obtained.
- In a similar way, experience of transcending makes it possible to fulfill desires and intentions with a minimum of effort: When the mind transcends, awareness become saturated with the level from which all the laws of nature begin to operate – the unified field. Having the support of this powerful level, desires can effortlessly meet with fulfillment.



CS401 MPP

The Stream API

Primitive Type Streams

Primitive Type Streams

- There are variations of Stream specifically designed for primitives: int, double, and long:
 - IntStream
 - DoubleStream
 - LongStream
- For primitive types short, char, byte, and boolean,
 - Use IntStream;
- For floats,
 - Use DoubleStream.

Creation Methods

- Are similar to those for Stream:

```
IntStream ints = IntStream.of(1, 2, 4, 8);  
IntStream ones = IntStream.generate(() -> 1);  
IntStream naturalNums = IntStream.iterate(1, n -> n+1);
```

Creating a Range

- `IntStream` (and also `LongStream`) have static methods **`range`** and **`rangeClosed`** that generate integer ranges with step size one:

```
//Upper bound is excluded  
IntStream zeroToNinetyNine = IntStream.range(0, 100);
```

```
//Upper bound is included  
IntStream zeroToHundred = IntStream.rangeClosed(0, 100);
```

Primitive to Object

- To convert a **primitive** type stream to an stream of objects, use the **boxed()** method:

```
Stream<Integer> integers = IntStream.range(0, 100).boxed();
```

Object to Primitive

- To convert an object stream to a primitive type stream, there are methods **mapToInt**, **mapToLong**, and **mapToDouble**

```
Stream<String> words = ...;  
IntStream lengths = words.mapToInt(String::length);
```

Converts **Stream<String>** to
IntStream (of lengths)

Main Differences to Object Streams

1. The **toArray** methods return primitive type arrays.
2. Methods that yield an **Optional** result return an **OptionalInt**, **OptionalLong**, or **OptionalDouble**
 - These classes are analogous to the `Optional` class, but they have methods **getAsInt**, **getAsLong**, and **getAsDouble** instead of the `get` method
3. There are methods `sum`, `average`, `max`, and `min` that return the sum, average, maximum, and minimum
 - These methods are not defined for object streams
 - Note that the functions `max` and `min` defined on an ordinary `Stream`, require a `Comparator` argument, and return an `Optional`



CS401 MPP

The Stream API

Lambda Libraries

Can Streams be Re-Used?

- Once a terminal operation has been called on a stream, the stream becomes unusable
 - If you do try to use it, you get an **IllegalStateException**
- But sometimes it would make sense to have a Stream ready to be used for multiple purposes

Example

- We have a **Stream<String>** that we might want to use for different purposes:

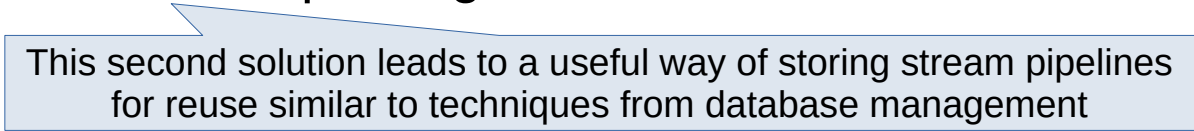
```
Folks.friends.stream().filter(name -> name.startsWith("N"));
```

- Typical Uses:
 - count the number of names obtained
 - output the names in upper case to a List

But once the stream has been used once,
we can't use it again!

Re-Use Techniques

1. One solution is to place the stream-creation code in a method and call it for different purpose.
 - See **Good** solution in package `lesson9.lecture.streamreuse`
2. Another solution is to capture all the free variables in the first approach as parameters of some kind of a Function (might be a BiFunction, TriFunction, etc, depending on the parameters).
 - See **Reuse** solution in package `lesson9.lecture.streamreuse`



This second solution leads to a useful way of storing stream pipelines for reuse similar to techniques from database management

Creating a Lambda Library

- Java 8 lets you perform queries to work with data in a Collection
 - The query style is similar to SQL queries.
- Database Problem:
 - You have a database table named Customer.
 - Return a collection of the names of Customers whose city of residence begins with the string “Ma”, arranged in sorted order.
- SQL Solution:
 - `SELECT name FROM Customer WHERE city LIKE 'Ma%' ORDER BY name`

Storing Queries

- We can store the query by naming it and replacing parameters with wildcards:

```
private final String SEARCH_CITY_IN_CUST = "SELECT name FROM Customer WHERE city  
LIKE '?' ORDER BY name";
```

- And then use it as:

```
Connection conn = DriverManager.getConnection();  
PreparedStatement stat = conn.prepareStatement(SEARCH_CITY_IN_CUST);  
stat.setString(1, "Ma%");
```

Similarly with Java Lambdas

- Java Problem:
 - You have a List of Customers.
 - Output to a list, in sorted order, the names of Customers whose city of residence begins with the string “Ma.”

```
List<String> listStr = list.stream()  
    .filter(cust -> cust.getCity().startsWith("Ma"))  
    .map(cust -> cust.getName())  
    .sorted().collect(Collectors.toList());
```

From Pipeline to Library Element

- How do you turn your solution into a Library Element?
 - Identify the parameters and treat them as arguments for some kind of function-type interface: **Function**, **BiFunction**, **TriFunction**, etc
- Parameters in our example (last slide):
 - An input list of type **List<Customer>**
 - A target string used to compare with name of city, of type **String**
 - Return type: a list of strings: **List<String>**

Library Element

- Our example can be stored in a BiFunction:

```
public class LambdaLibrary {  
    public static final BiFunction<List<Customer>, String, List<String>> NAMES_IN_CITY  
        = (list, searchStr) -> list.stream()  
            .filter(cust -> cust.getCity()  
                .startsWith(searchStr))  
            .map(cust -> cust.getName())  
            .sorted()  
            .collect(Collectors.toList());  
}
```

- We can then use our library element like:

```
List<String> listStr =  
    LambdaLibrary.NAMES_IN_CITY.apply(list, "Ma");
```

See: lesson9.lecture.lambdalibrary.

Where list is your list



CS401 MPP

The Stream API

Closing Remarks

Connecting the Parts of Knowledge With the Wholeness of Knowledge

1. Prior to the release of Java 8, extracting or manipulating data in one or more lists or other Collection classes involved multiple loops and code that is often difficult to understand.
2. With the introduction of lambdas and streams, Java 8 makes it possible to create compact, readable, reusable expressions that accomplish list-processing tasks in a very efficient way. These can be accumulated in a Lambda Library.
3. **Transcendental Consciousness** is the field that underlies all thinking and creativity, and, ultimately, all manifest existence.
4. **Impulses Within the Transcendental Field.** The hidden self-referral dynamics within the field of pure intelligence provides the blueprint for emergence of all diversity. This blueprint is formed from compact expressions of intelligence are coherently arranged in the first sprouting of existence.
5. **Wholeness Moving Within Itself.** In Unity Consciousness, the fundamental forms out of which manifest existence is structured are seen to be vibratory modes of one's own consciousness.

